

Heap and Heap Sort

22 April 2026 19:22

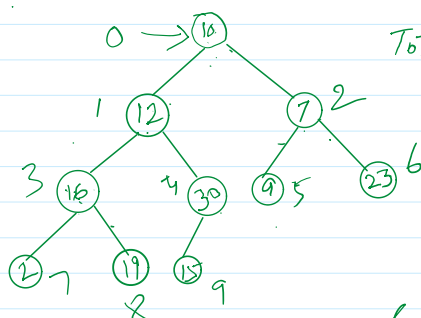
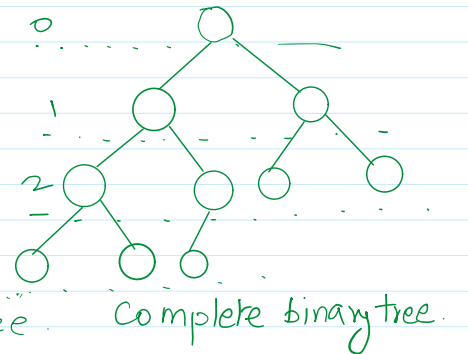
What is Complete binary Tree?

A Complete-Binary Tree is a binary tree in which:

- All levels are completely filled except possibly the last level.
- The last level is filled from left to right without gaps.

Key properties

- Nodes are filled level by level.
- Last level nodes are as left as possible.
- No gaps between nodes.



height of tree → $2^n - 1$
Total no. elements = $2^4 - 1 = 16 - 1 = 15$

Level 0 → full
Level 1 → full
Level 2 → full
Last level → filled from left (no gaps)

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
10	12	7	16	30	9	23	2	7	19	15				

feed the binary tree in the array first, i.e., you place nodes level by level (breadth-first), left-to-right. The structure of a CBT guarantees there are no gaps, so mapping is direct.

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
10	12	7	16	30	9	23	2	7	19	15				
		P ↑		↑ ↑										

For any node at index i :
Left child → $2 \times i + 1$
Right child → $2 \times i + 2$
Parent → $\lfloor (i-1)/2 \rfloor$

$i = 0$
parent = $\lfloor (0-1)/2 \rfloor = 0 \Rightarrow 10$
left child = $2 \times 0 + 1 = 1 \Rightarrow 12$
right child = $2 \times 0 + 2 = 2 \Rightarrow 7$

$i = 1 \Rightarrow 12$
parent → $\lfloor (1-1)/2 \rfloor \rightarrow 0 = 10$
left → $\lfloor 2 \times 1 + 1 \rfloor \rightarrow 3 = 16$
right → $\lfloor 2 \times 1 + 2 \rfloor \rightarrow 4 = 30$

$i = 2 \Rightarrow 7$
parent → $\lfloor (2-1)/2 \rfloor \Rightarrow 0 \Rightarrow 10$
left → $2 \times 2 + 1 \Rightarrow 5 \Rightarrow 9$
right → $2 \times 2 + 2 \Rightarrow 6 \Rightarrow 23$

$i = 3 \rightarrow 16$
parent $(3-1)/2 \Rightarrow 1 \Rightarrow 12$
left $(2 \times 3) + 1 \Rightarrow 7 \Rightarrow 2$
right $(2 \times 3) + 2 \Rightarrow 8 \Rightarrow 19$

Heap (memory + data structure)

What is Heap Memory?

In Java, the heap is a runtime memory area where:

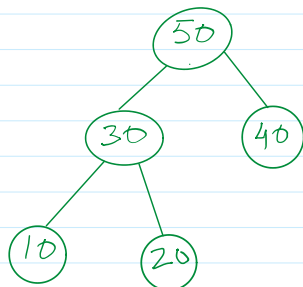
- Objects are created.
- Instance variables are stored.
- Managed by garbage collection.(GC)

What is a Heap?

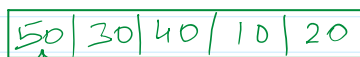
A heap is a complete binary tree that satisfies the heap property.

Types:

- i. Max heap => Parent $\geq$ children
- ii. Min heap => Parent $\leq$ children



← Every parent is greater than its children.



↑ maximum value.
in max-heap

$$\begin{aligned} \text{parent} &= (i-1)/2 \\ \text{left-child} &= 2*i + 1 \\ \text{right-child} &= 2*i + 2 \end{aligned}$$

Heap Sort

Heap sort is a comparison based sorting algorithm that:

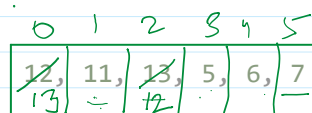
- Builds a Max Heap
- Repeatedly extracts the maximum element.
- Places it at the end.

Steps

1. Convert array into Max heap.
2. Swap root (largest) with last element.
3. Reduce heap size.
4. Heapify again.
5. Repeat

```

package HeapInJava;
import java.util.Arrays;
public class HeapSort {
    void heapify(int arr[], int n, int i){
        int largest = i;
        int left = 2 * i + 1;
        int right = 2 * i + 2;
        if(left < n && arr[left] > arr[largest]){
            largest = left;
        }
        if(right < n && arr[right] > arr[largest]){
            largest = right;
        }
        if(largest != i){
            int t = arr[i];
            arr[i] = arr[largest];
            arr[largest] = t;
            heapify(arr, n, largest);
        }
    }
    void heapSort(int arr[]){
        int n = arr.length;
        // Step one: build Max heap
        for (int i = n/2 - 1; i >= 0; i--) {
            heapify(arr, n, i);
        }
        // Step 2: extract elements.
  
```

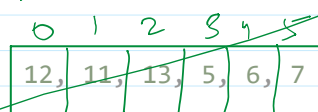


$n = 6$
 $i = 2$
 $\text{largest} = 2$
 $\text{left} = 5$
 $\text{right} = 6$

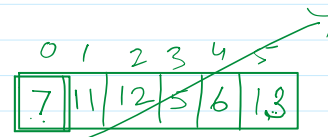
$n = 6$
 $i = 1$
 $\text{largest} = 1$
 $\text{left} = 3$
 $\text{right} = 4$

$n = 6$
 $i = 0$
 $\text{largest} = 0$
 $\text{left} = 1$
 $\text{right} = 2$
 $t = 12$

$n = 6$
 $i = 2$
 $\text{largest} = 2$
 $\text{left} = 5$
 $\text{right} = 6$



$n = (6)$



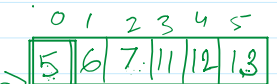
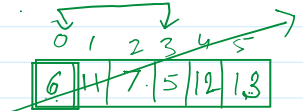
```

    heapify(arr, n, i);
}
// Step 2: extract elements.
→ for (int i = n - 1; i >= 0; i--) {
    → int t = arr[0];
    arr[0] = arr[i];

    arr[i] = t;
    // Step 3: call heapify on the reduced heap
    → heapify(arr, i, 0);
}
}
public static void main(String[] args) {
    int arr[] = {12, 11, 13, 5, 6, 7};
    HeapSort hs = new HeapSort();
    System.out.println("\nBefore sorting: " + Arrays.toString(arr));
    hs.heapSort(arr);
    System.out.println("\nAfter sorting: " + Arrays.toString(arr));
}
}

```

$n = 6$
 $i = 2 + 0 - 1 = 1$ ~~2~~ 4 3
~~heapify(arr, 6, 2)~~
~~heapify(arr, 1, 1)~~
~~heapify(arr, 1, 0)~~
 $t = 13$ 12 11
~~heapify(arr, 5, 0), heapify(arr, 4, 0)~~
~~heapify(arr, 3, 0)~~



do the day run:

Output:

```

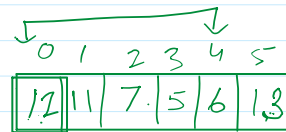
Before sorting: [12, 11, 13, 5, 6, 7]
After sorting: [5, 6, 7, 11, 12, 13]

```

```

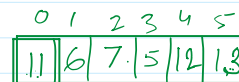
void heapify(int arr[], int n, int i){
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;
    if(left < n && arr[left] > arr[largest]){
        largest = left;
    }
    → if(right < n && arr[right] > arr[largest]){
        largest = right;
    }
    if(largest != i){
        → int t = arr[i];
        arr[i] = arr[largest];
        arr[largest] = t;
        heapify(arr, n, largest);
    }
}
}

```



$n = 5$
 $i = 0$
 $largest = 0$ 2
 $left = 1$
 $right = 2$
 $t = 7$
~~heapify(arr, 5, 2)~~

$n = 5$
 $i = 2$
 $largest = 2$
 $left = 5$
 $right = 6$



$n = 4$
 $i = 0$
 $largest = 0$ 1
 $left = 1$
 $right = 2$
 $t = 6$
~~heapify(arr, 4, 1)~~

$n = 4$
 $i = 1$
 $largest = 1$
 $left = 3$
 $right = 4$